Task 3: Session 3 (AIML) - Assignment Questions

Name: Mihir Sopan Patil

Domain: AIML

College & Branch Name: Zeal Polytechnic, Diploma In AIML

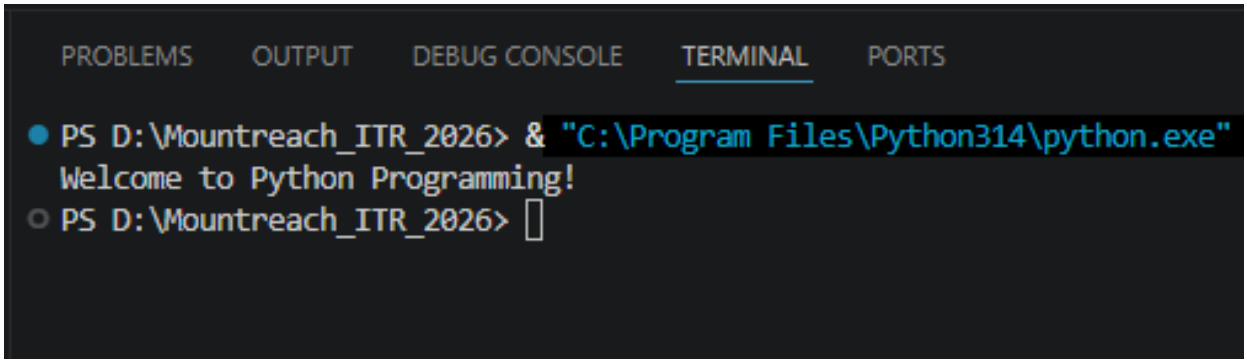
1. Basic Function

Write a function named welcome() that prints the message: "Welcome to Python Programming!"

Call the function to execute it.

Add a comment explaining why functions are useful.

Output:



Program Code:

```
# Functions Are Organized, Reusable Blocks Of Code That Perform Specific Task

# Defining A Function
def welcome():
    print("Welcome to Python Programming!")
# Calling A Function
welcome()
```

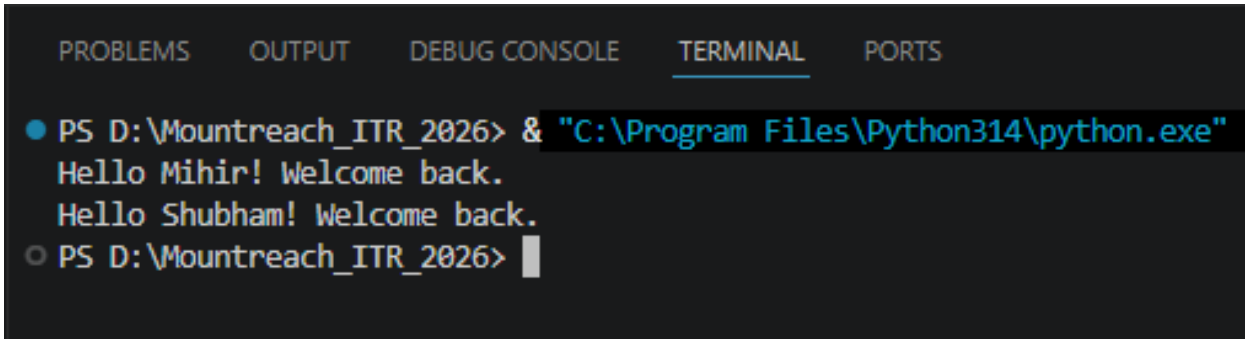
2. Function With Parameter

Create a function named greet(name) that takes one parameter name and prints: "Hello, [name]! Welcome back."

Call the function with your own name.

Also call it with a different name to show reusability.

Output:



Program Code:

```
# Defining A Function
def greet(name):
    print("Hello", name + "! Welcome back.")

greet("Mihir")       # Call With Your My Name
greet("Shubham")     # Call With A Different Name To Show Reusability
```

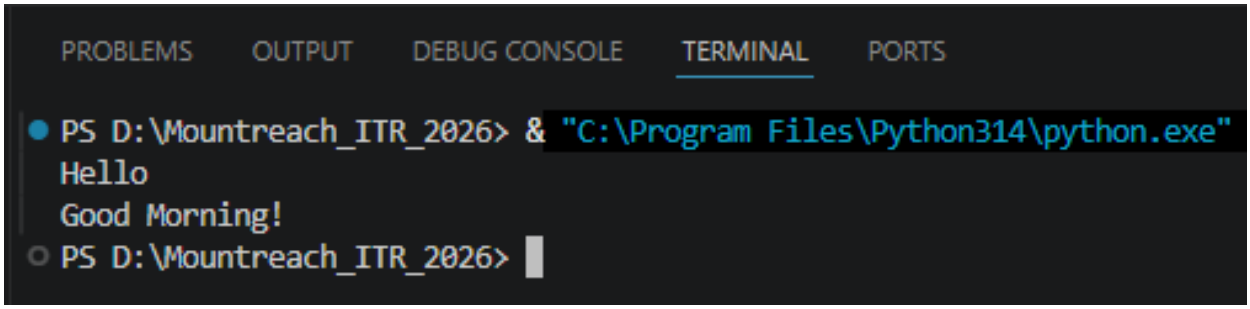
3. Default Parameter

Write a function show_message(message="Hello") that prints the message.

- Call the function without passing any argument.
- Call the function again by passing a different message.

Explain the benefit of default parameters in a comment.

Output:



Program Code:

```
# Default Parameter Allows Function To Use Predefined Values If No Argument Is
Passed

# Defining A Function
def show_message(message="Hello"):
    print(message)

show_message()                  # Uses Default Value "Hello"
show_message("Good Morning!")   # OverridesWith Passed Argument "Good Morning!"
```

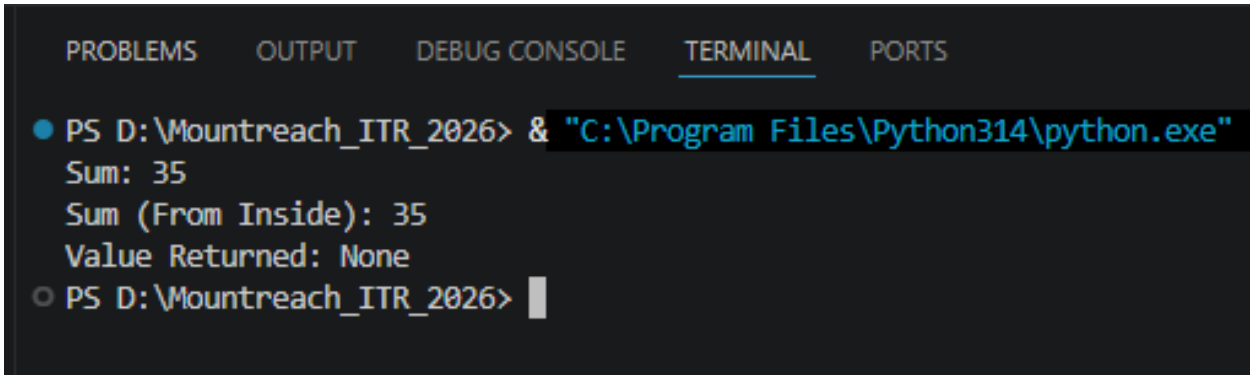
4. Function With Multiple Parameters & Return

Create a function calculate_sum(a, b) that takes two numbers and **returns** their sum.

- Call the function with two numbers and store the result in a variable.
- Print the result.

Also demonstrate the difference between print() inside the function and using return.

Output:



Program Code:

```
# Defining A Function
def calculate_sum(a, b):
    return a + b   # return Sends The Value Back To The Caller

result = calculate_sum(10, 25)
print("Sum:", result)

# Difference Between print() Inside A Function & return:
# print(): Displays The Value But Gives Nothing Back To The Caller.
# return: Sends The Value Back To The Caller So They Can Store/Use It Further.

# Defining A Function
def calculate_sum_with_print(a, b):
    print("Sum (From Inside):", a + b)   # Prints But Returns None

val = calculate_sum_with_print(10, 25)
print("Value Returned:", val)            # None, Because No Return Statement
```

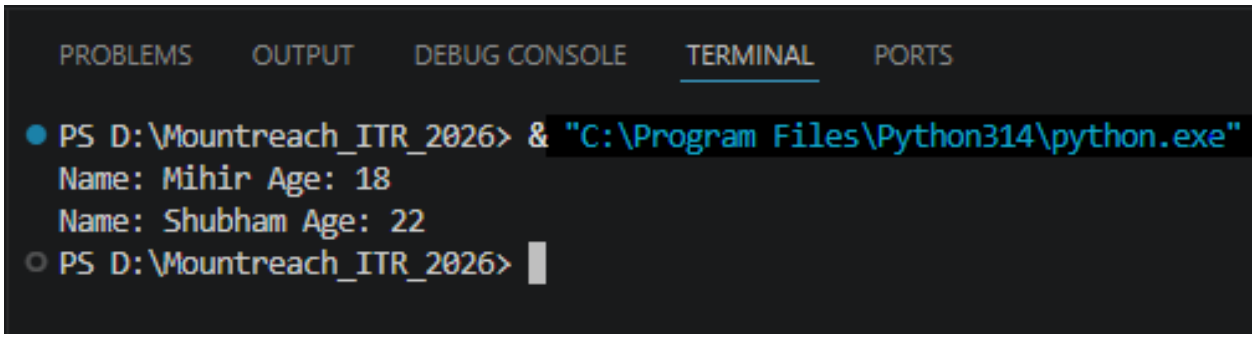
5. Positional vs Keyword Arguments

Write a function student_info(name, age) that prints the name and age.
Call this function **twice**:

- Using positional arguments.
- Using keyword arguments.

Add comments explaining the difference.

Output:



Program Code:

```
# Define A Function
def student_info(name, age):
    print("Name:", name, "Age:", age)

# Positional Arguments: Values Are Matched To Parameters By Their Position.
student_info("Mihir", 18)

# Keyword Arguments: Values Are Matched By Parameter Name, Order Doesn't Matter.
student_info(age=22, name="Shubham")
```

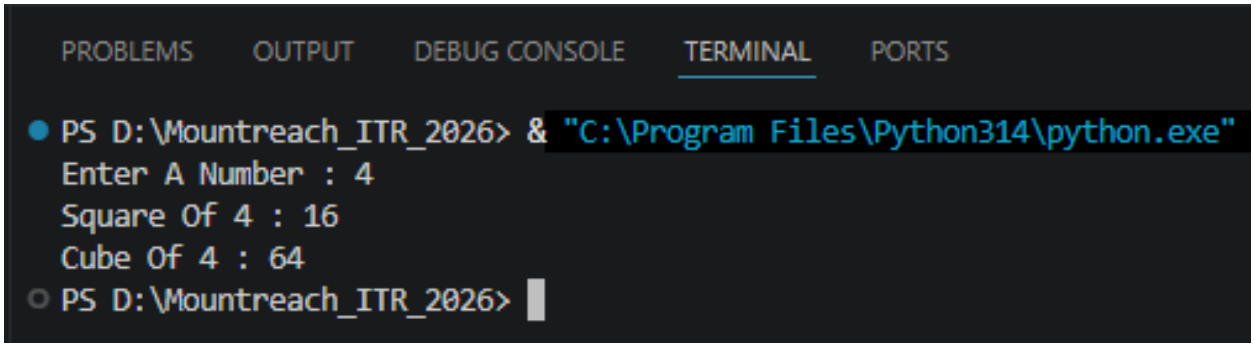
6. Function With Return Value

Write a function square(num) that returns the square of the given number.

Write another function cube(num) that returns the cube of the number.

In the main program, take a number as input from the user and print its square and cube using these functions.

Output:



Program Code:

```
# Defining Functions
def square(num):
    return num ** 2 # Returns The Square Of The Number

def cube(num):
    return num ** 3 # Returns The Cube Of The Number

number = int(input("Enter A Number : "))
print("Square Of", number, ":",square(number))
print("Cube Of", number, ":",cube(number))
```

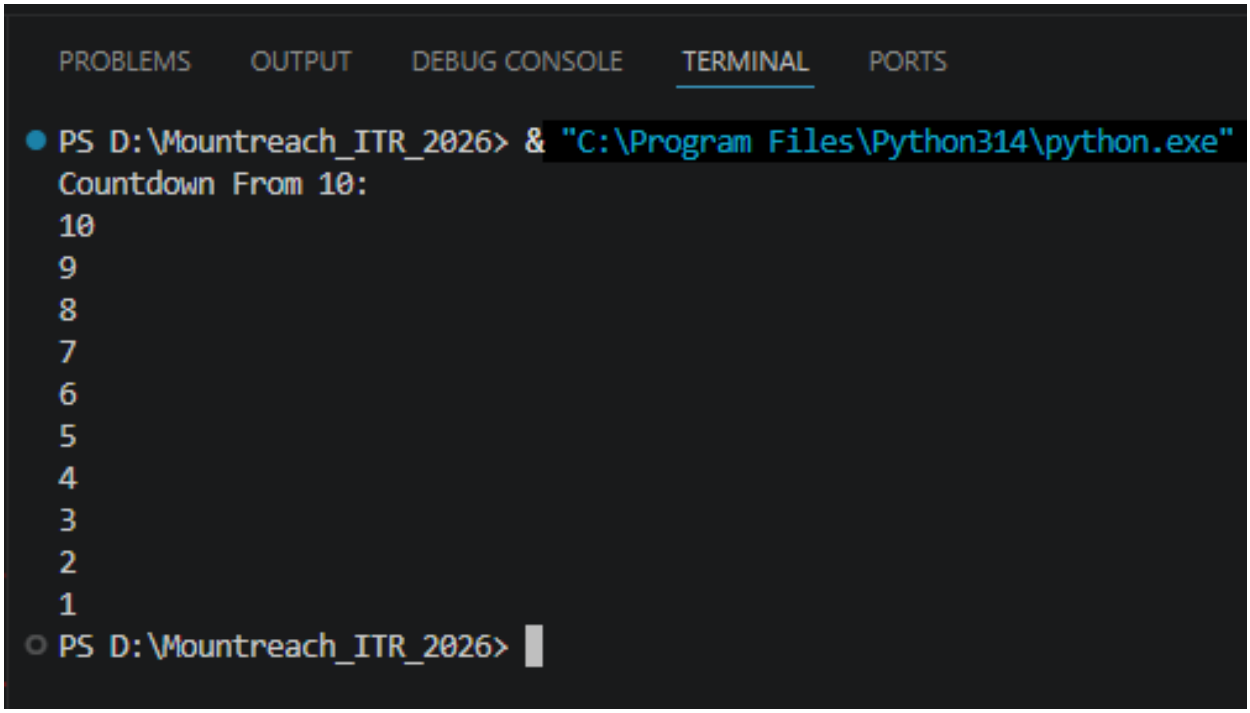
7. Recursion - Basic

Write a recursive function countdown(n) that prints numbers from n down to 1.

Call countdown(10).

Make sure to include the base case (stopping condition).

Output:



Program Code:

```
# Define A Recursive Function To Print A Countdown From A Given Number
def countdown(n):
    # Base Case: Stop Recursion When n Reaches 0
    if n <= 0:
        return
    print(n)
    countdown(n - 1)   # Recursive Call With n Reduced By 1

print("Countdown From 10:")
countdown(10)
```

8. Recursion - Factorial

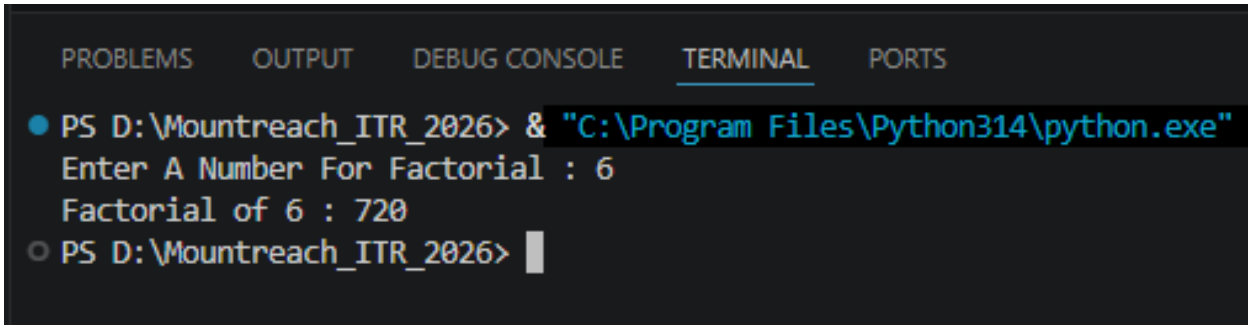
Write a recursive function factorial(n) that calculates and **returns** the factorial of a number.

Example: factorial(5) should return 120.

Test it with input from the user and print the result.

Add a comment explaining the base case.

Output:



Program Code:

```
def factorial(n):
    # Base Case: Factorial Of 0 Or 1 Is 1
    if n <= 1:
        return 1
    return n * factorial(n - 1)   # Recursive Call

n = int(input("Enter A Number For Factorial : "))
print("Factorial of", n, ":", factorial(n))
```

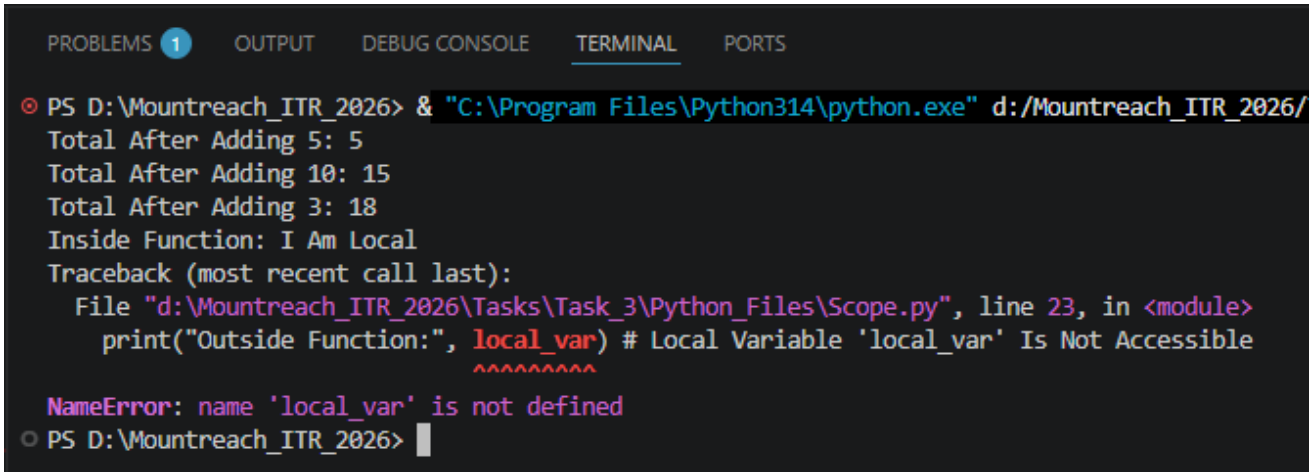
9. Scope - Local vs Global

Write a program to demonstrate scope:

- Create a global variable total = 0
- Write a function add_value(x) that adds x to total (use the global keyword).
- Call the function multiple times and print the value of total after each call.

Also try to print a local variable outside its function to show it doesn't work.

Output:



Program Code:

```
total = 0   # Global Variable

def add_value(x):
    global total          # Declare We Want To Modify The Global 'total'
    total = total + x

add_value(5)
print("Total After Adding 5:", total)

add_value(10)
print("Total After Adding 10:", total)

add_value(3)
print("Total After Adding 3:", total)

# Demonstrating That A Local Variable Cannot Be Accessed Outside Its Function:
def demo_local():
    local_var = "I Am Local"
    print("Inside Function:", local_var)
```

```
demo_local()

print("Outside Function:", local_var) # Local Variable 'local_var' Is Not Accessible
```

10. Debugging & Understanding

Create a **Simple Number Analyzer** program using functions:

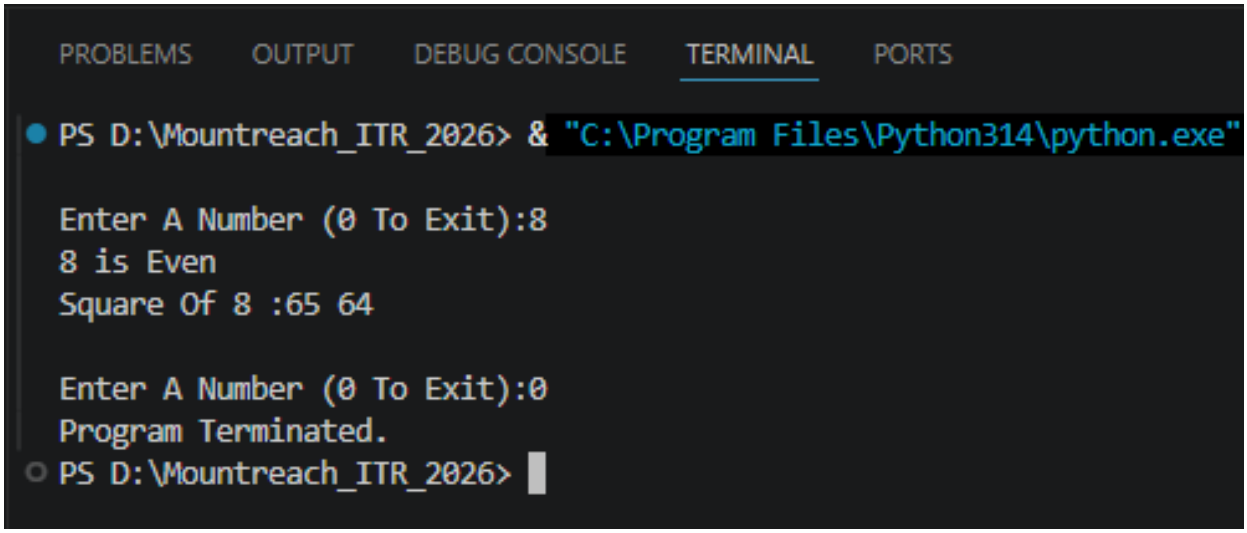
Write the following functions:

- get_number() → takes input from user and returns the number.
- is_even(num) → returns True if number is even, else False.
- power(base, exp=2) → returns base raised to exponent (use default argument).
- show_result(num) → prints whether the number is even/odd and its square.

Use a while loop to repeatedly ask the user for numbers until they enter 0.

Use recursion **only** if needed for one small part.

Output:



Program Code:

```
# Function To Get A Number From The User
def get_number():
    return int(input("\nEnter A Number (0 To Exit):"))

# Function To Check If A Number Is Even
def is_even(num):
    return num % 2 == 0

# Function To Calculate Power (Default Exponent Is 2)
def power(base, exp=2):
    return base ** exp
```

```
# Function To Display The Result
def show_result(num):
    if is_even(num):
        print(num, "is Even")
    else:
        print(num, "is Odd")

    print("Square Of", num, ":65", power(num))


while True:
    number = get_number()

    if number == 0:
        print("Program Terminated.")
        break

    show_result(number)
```